



Numerical Methods for PDEs | WS 2025/26

Programming Exercise 3 | 06.01.2026

Notes concerning the submission of a programming exercise (read carefully!):

- The programming exercises should be completed in **your Moodle group**.
- You can use whatever programming language you like.
- Please make an appointment for your group attestation using the **Moodle appointment planer**. Attestations will take place 23.01.2026, and you can register until 20.01.2026.
- Every group member needs to take an active role during the attestation.

Finite Volume Method

Given the following scalar conservation law

$$u_t + (f(u))_x = 0,$$

with periodic initial condition

$$u(x, 0) = u_0(x) = \begin{cases} 1, & x \in [-1, -1/2), \\ 0, & x \in [-1/2, 1/2), \\ 1, & x \in [1/2, 1). \end{cases}$$

Let us use following form of Finite Volume Method

$$u_j^{n+1} = u_j^n + \frac{\Delta t}{\Delta x} (F(u_{j-1}^n, u_j^n) - F(u_j^n, u_{j+1}^n))$$

with following numerical fluxes F :

- **naive:**

$$F(u_l, u_r) = \frac{1}{2}(f(u_l) + f(u_r))$$

- **Lax-Friedrichs:**

$$F(u_l, u_r) = \frac{1}{2}(f(u_l) + f(u_r)) - \frac{\Delta x}{2\Delta t}(u_r - u_l)$$

- **Lax-Wendroff:**

$$F(u_l, u_r) = \frac{1}{2}(f(u_l) + f(u_r)) - \frac{\Delta t}{2\Delta x} a^2(u_l, u_r)(u_r - u_l) \quad \text{where} \quad a(u_l, u_r) = \frac{f(u_l) - f(u_r)}{u_l - u_r}$$

1) Implement presented numerical schemes for fluxes

$$f(u) = 2u \quad (\text{advection equation}),$$

$$f(u) = \frac{1}{2}u^2 \quad (\text{Burgers equation}).$$

Make your program take (at least) take following input: number of grid nodes N , end time T , initial condition u_0 and which numerical flux shall be used. Plot your solution.

Hint:

- Make sure that CFL condition is met:

$$\Delta t \leq \frac{\Delta x}{\max_u |f'(u)|}$$

- As a rule, the evaluations of the numerical flow function at the cell boundaries represent a large part of the total computational effort of a FV solver. Therefore, try to get along with as few flow function evaluations as possible.
- For the Burgers equation, the exact solution for the smooth initial condition cannot be explicitly specified. You can implement the exact solution with the help of the minimization function `fminbound` from the *scipy.optimize* library (you need to adapt the following snippet for your code):

```
xmin = fminbound(lambda x: u0(x), x0, x1)
u0min = u0(xmin)
xmax = fminbound(lambda x: -u0(x), x0, x1)
u0max = u0(xmax)
u_ex = lambda x, t:
    fminbound(lambda u: (u-u0(x-u*t)**2), u0min, u0max)
```

In this case u_0 denotes the initial condition, and x_0 and x_1 are the left and right interval indicate a border.

- 2) Examine the convergence order of the procedures with respect to the L^1 -norm in a loglog plot. In addition to the error of the numerical approximation $u_{\Delta t}$ to the exact solution u on grid points, consider the error of the cell-wise constant approximation of the exact solution u by the cell mean values (denoted by $P_0 u$), i.e.

$$\|u_{\Delta t} - u\|_{L^1([-1,1] \times [0,T])} \quad \text{and} \quad \|u_{\Delta t} - P_0 u\|_{L^1([-1,1] \times [0,T])}.$$

To do this, use the discontinuous initial condition specified above, as well as the smooth initial condition

$$u_0(x) = \sin(\pi x), \quad x \in [-1, 1].$$

In the case of the Burgers equation, select the end time T such that the characteristics do not intersect. Compare convergence plots. What convergence order do you expect from the theory for the respective methods??

Hint:

- The package *scipy.integrate* offers a number of different numerical integration methods. For the purpose of this exercise, you may use

```
integral_approx = fixed_quad(lambda x: f(x), x0, x1, n=5)[0]
```

where $f(x)$ is the integrand, x_0 and x_1 specify the interval of integration and $n = 5$ specifies that a fixed fifth order quadrature scheme is used.

- Other numerical integration schemes from this package, e.g.

```
quad(lambda x: f(x), x0, x1)
```

for an adaptive one dimensional integration or

```
dblquad(lambda x,y: f(x,y), y0, y1,
         lambda x: x0(x), lambda x: x1(x))
```

for an adaptive two dimensional integration both turn out to be too slow for our purposes. Therefore it is recommended to use the fixed order one dimensional method `fixed_quad(...)` in order to define your own two dimensional numerical integration method. Alternatively, you can of course also implement your own quadrature code.

10+10=20 Points

Riemann Problem for Systems

Consider the one dimensional non-linear Euler equations

$$\begin{aligned}\frac{\partial}{\partial t}\rho + \frac{\partial}{\partial x}(\rho v) &= 0, \\ \frac{\partial}{\partial t}(\rho v) + \frac{\partial}{\partial x}(\rho v^2 + p) &= 0, \\ \frac{\partial}{\partial t}E + \frac{\partial}{\partial x}(v(E + p)) &= 0.\end{aligned}$$

where ρ , v , p and E are the density, velocity, pressure and total energy. We will assume E and p to be related as

$$p = (\gamma - 1) \left(E - \rho \frac{v^2}{2} \right),$$

where, for mono-atomic gases, we can choose $\gamma = 1.4$. The speed of sound can be given as $c = \sqrt{\frac{\gamma p}{\rho}}$. We will assume $(x, t) \in [0, 1] \times [0, 0.25]$. For the Riemann problem we will consider the discontinuity in the initial data to be located at $x = 0.5$. The two sets of initial data which we will consider have been given in Table-1.

Test-Case	ρ_L	v_L	p_L	ρ_R	v_R	p_R
1	1.0	0	1.0	0.125	0	0.1
2	1.0	-1	0.4	1.0	1	0.4

Table 1: Initial Conditions for Different Test Cases

Solve the Euler equations for the two different initial data using a finite volume scheme based upon the HLL flux. In the conservative form, a finite volume scheme based upon the HLL flux can be given as

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x} (F(u_i^n, u_{i+1}^n) - F(u_{i-1}^n, u_i^n)) \quad (1)$$

where

$$F(u_i^n, u_{i+1}^n) = \begin{cases} f(u_i^n), & 0 \leq a_L \\ \frac{a_R f(u_i^n) - a_L f(u_{i+1}^n) + a_L a_R (u_{i+1}^n - u_i^n)}{a_R - a_L}, & a_L \leq 0 \leq a_R \\ f(u_{i+1}^n), & 0 \geq a_R \end{cases} \quad (2)$$

with $a_L = \min(\lambda(f'(u_i)), \lambda(f'(u_{i+1})))$ and $a_R = \max(\lambda(f'(u_i)), \lambda(f'(u_{i+1})))$; $\lambda(f'(u))$ represent the eigenvalues of the Jacobian $f'(u)$.

As boundary conditions, you may use a zero-order extrapolation.

Hint:

- You can compute the analytical solutions for the eigenvalues of the Jacobian $f'(u)$ by hand. Doing so is easier in primitive variables (ρ, u, p) than in conservative variables $(\rho, \rho u, E)$. The eigenvalues however, are the same.
- Since we are investigating discontinuous initial conditions, your solver should be formulated in conservative variables $(\rho, \rho u, E)$.

20 Points